

Каналы (System.Threading.Channels): современный асинхронный producer/consumer

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?





Гладченко Николай Михайлович

Старший разработчик

- 12 лет в разработке (коммерческая разработка + R&D).
- Архитектура микросервисов для Mercedes-Benz Group, разработка SPA-фреймворков, интеграция LLM/ML моделей.
- Красный диплом СПбГЭТУ, диплом TU Ilmenau Ingenieurinformatik
- Опыт работы в мультиязычных командах (Rus/Eng/Deu).

 @platonicum

 glad4enkonm@gmail.com



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **OTUS**
C#-in-depth-2025-08



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу



Маршрут вебинара

Зачем

Основы `System.Threading.Channel`

Жизненный цикл канала и его закрытие

Типы каналов и сценарии использования

Практика

Основные тезисы



Цели вебинара

К концу занятия вы сможете

1. Реализовывать паттерн "Производитель-потребитель" с помощью Channels;
2. Строить эффективные асинхронные конвейеры для разделения I/O-операций и логики обработки;
3. Управлять жизненным циклом канала и обрабатывать его закрытие.



Смысл

Это нужно уметь чтобы:

1. Освоить современный асинхронный паттерн

2. Строить отказоустойчивые конвейеры

3. Применять Channels для упрощения асинхронного взаимодействия между



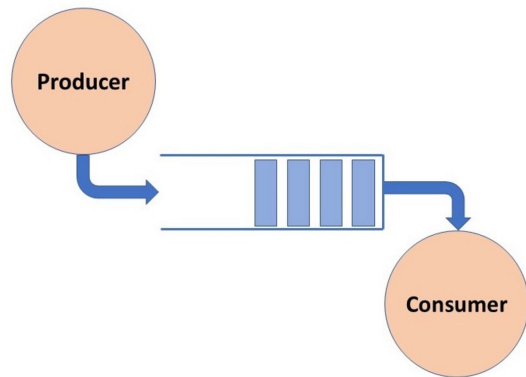
System.Threading.Channels

Паттерн "Производитель- потребитель"



Паттерн "Производитель-потребитель"

- Производитель (Producer): Компонент, который генерирует данные или задачи.
- Потребитель (Consumer): Компонент, который обрабатывает эти данные или задачи.
- Общая очередь/буфер: Механизм для безопасного обмена данными между производителем и потребителем.
- Ключевая цель: Разделение логики генерации и обработки данных, а также управление нагрузкой и потоком.



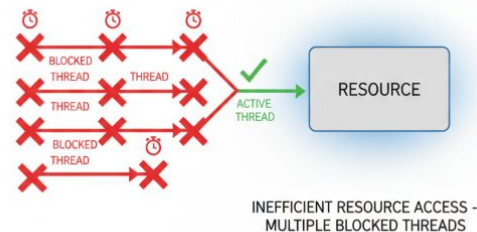
Традиционные Подходы

Синхронные Очереди

- `ConcurrentQueue<T>`:
 - не асинхронное решение;
- `BlockingCollection<T>`:
 - Предоставляет потокобезопасную коллекцию для Producer/Consumer.
 - Основана на `IProducerConsumerCollection<T>`.
 - Методы `Add()` и `Take()` являются блокирующими.

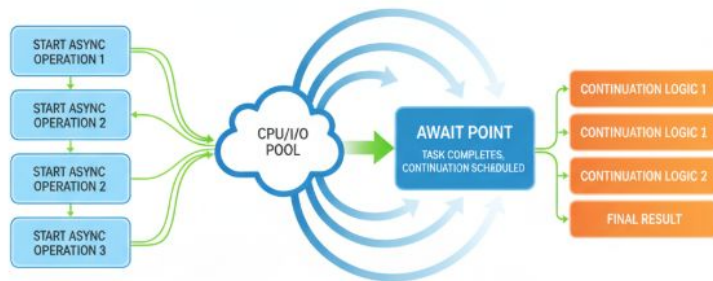
Проблемы блокирующих подходов в асинхронных сценариях

- **Блокировка потоков:** При `BlockingCollection.Take()` или `BlockingCollection.Add()` (если очередь заполнена) поток ожидает данных или места.
- **Контекстные переключения:** Операционная система должна постоянно переключаться между заблокированными и активными потоками, что дорого.
- **Масштабируемость:** Ограничение на количество потоков. Создание множества потоков для каждого производителя/потребителя неэффективно и может привести к дедлокам.
- **Асинхронные операции ввода/вывода (I/O):** Наши производители или потребители часто выполняют `await`-операции (сеть, диск). Блокировка потока на `Take()` или `Add()` при ожидании `await` может привести к нежелательному поведению и снижению производительности.



Асинхронная проблема, асинхронное решение

- Нужен асинхронный примитив для реализации Producer/Consumer.
- Примитив, который позволяет асинхронно ожидать доступности данных или места в очереди, не блокируя потоки.
- Это позволяет создавать эффективные асинхронные конвейеры обработки данных.



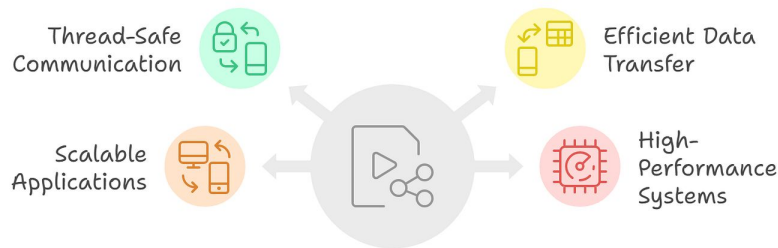
`System.Threading.Channels` — это библиотека, которая предоставляет типы для асинхронной передачи данных между производителями и потребителями.



System.Threading.Channels

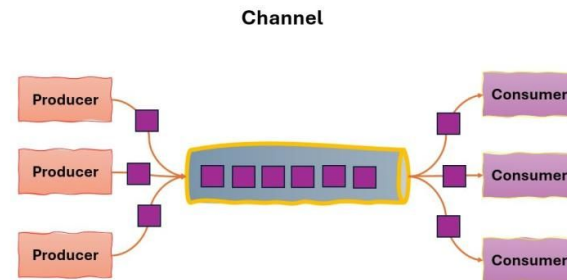
- Введены в .NET Core 3.0 (доступны как NuGet-пакет для более старых версий).
- Предоставляют набор высокопроизводительных, асинхронных примитивов для работы с паттерном "Производитель-Потребитель".
- Позволяют асинхронно записывать (WriteAsync) и читать (ReadAsync) элементы, используя Task-based API.
- Не блокируют потоки при ожидании данных или места.
- Подходят для построения асинхронных конвейеров, обработки потоковых данных и внутренних очередей сообщений.

C# Channels Benefits



Ключевые преимущества Channels

- **Асинхронность:** Не блокируют потоки, что критично для высокопроизводительных приложений.
- **Эффективность:** Минимальные накладные расходы, оптимизированы для асинхронной работы.
- **Простота использования:** Чистый и интуитивно понятный API (Writer, Reader).
- **Гибкость:** Поддержка ограниченных (bounded) и неограниченных (unbounded) каналов с различными стратегиями управления.
- **Управление жизненным циклом:** Четкие механизмы для завершения работы каналов.



Основы `System.Threading.Channels`



System.Threading.Channels.Channel<T>

- Абстрактный базовый класс, представляющий канал между производителями и потребителями.
- Содержит две основные части:
 - Writer (тип `ChannelWriter<T>`): для записи элементов в канал.
 - Reader (тип `ChannelReader<T>`): для чтения элементов из канала.
- Создание канала: Используем статические фабричные методы `Channel.CreateUnbounded<T>()` или `Channel.CreateBounded<T>()`.
 - `Channel.CreateUnbounded<T>()`: Создает канал без ограничения по размеру.
 - `Channel.CreateBounded<T>(options)`: Создает канал с заданным максимальным количеством элементов.

System.Threading.Channels.Channel<T>

```
var channel = Channel.CreateBounded<Coordinates>(
    new BoundedChannelOptions(1_000)
    {
        SingleWriter = true,
        SingleReader = false,
        AllowSynchronousContinuations = false,
        FullMode = BoundedChannelFullMode.DropWrite
    });
```

```
var channel = Channel.CreateBounded<Coordinates>(1);
```

```
var channel = Channel.CreateUnbounded<Coordinates>(
    new UnboundedChannelOptions
    {
        SingleWriter = false,
        SingleReader = false,
        AllowSynchronousContinuations = true
    });
```

```
var channel = Channel.CreateUnbounded<Coordinates>();
```

ChannelWriter<T>

- Представляет собой сторону канала, предназначенную для записи элементов.
- Производитель использует методы ChannelWriter<T> для добавления данных в канал.
- Получить ChannelWriter<T> можно через свойство Channel<T>.Writer.



Основные методы ChannelWriter<T>

- **WriteAsync(T item):** Асинхронная операция записи.
 - Возвращает ValueTask, которая завершается, когда элемент успешно добавлен в канал.
 - Если канал ограничен и полон, эта операция асинхронно ждет, пока место не освободится (не блокирует поток!).

Основные методы ChannelWriter<T>

- TryWrite(T item): Синхронная попытка записи.
 - Возвращает true, если элемент был немедленно добавлен в канал.
 - Возвращает false, если канал полон (для ограниченных) или не может принять элемент по другой причине (например, завершен).
 - Не блокирует и не ждет.

Основные методы ChannelWriter<T>

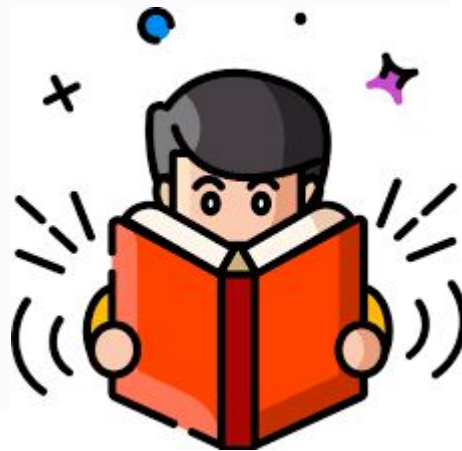
- **WaitToWriteAsync(CancellationToken cancellationToken = default):**
 - Возвращает ValueTask<bool>, которая завершается, когда в канале появляется место для записи.
 - true, если можно записать; false, если канал завершен и больше не принимает записи.
 - Полезно, если вы хотите сначала проверить возможность записи, а затем использовать TryWrite.

Основные методы ChannelWriter<T>

- Complete(Exception? error = null):
 - Сигнализирует, что производитель завершил свою работу и больше не будет добавлять элементы.
 - После вызова Complete(), WriteAsync и TryWrite будут завершаться с ошибкой (ChannelClosedException) или возвращать false.
 - Важно вызывать Complete() для корректной обработки завершения потребителями.

ChannelReader<T>

- Представляет собой сторону канала, предназначенную для чтения элементов.
- Потребитель использует методы ChannelReader<T> для получения данных из канала.
- Получить ChannelReader<T> можно через свойство Channel<T>.Reader.



Основные методы ChannelReader<T>

- ReadAsync(CancellationToken cancellationToken = default):
 - Асинхронная операция чтения.
 - Возвращает ValueTask<T>, которая завершается, когда элемент доступен.
 - Если канал пуст, эта операция асинхронно ждет, пока элемент не появится (не блокирует поток!).
 - Если канал завершен (Complete()) и пуст, выбрасывает ChannelClosedException.



Основные методы ChannelReader<T>

- TryRead(out T item):
 - Синхронная попытка чтения.
 - Возвращает true и извлекает элемент, если он немедленно доступен.
 - Возвращает false, если канал пуст или не может предоставить элемент.
 - Не блокирует и не ждет.

Основные методы ChannelReader<T>

- `WaitToReadAsync(CancellationToken cancellationToken = default):`
 - Возвращает `ValueTask<bool>`, которая завершается, когда в канале появляются элементы для чтения.
 - `true`, если можно читать; `false`, если канал завершен и пуст.
 - Полезно, если вы хотите сначала проверить доступность, а затем использовать `TryRead`.

Основные методы ChannelReader<T>

- Completion:
 - Свойство типа Task, которое завершается, когда канал завершен (Complete()) и все элементы были прочитаны.
 - Если Complete() был вызван с исключением, Completion.Status будет Faulted.
 - Полезно для ожидания полного завершения обработки каналом.

Использование await foreach для удобного чтения

- Все ChannelReader<T> в System.Threading.Channels реализуют IEnumerableAsync<T>.
- Значительно упрощает код потребителя. Автоматически асинхронно ждет элементы и завершается, когда канал Complete() и пуст.

```
Console.WriteLine("Потребитель начинает чтение...");
await foreach (var item in channel.Reader.ReadAllAsync())
{
    Console.WriteLine($"Получено: {item}");
}
Console.WriteLine("Потребитель завершил чтение (канал пуст и завершен).");
```



ПРИМЕР

Вопросы для проверки

1. Объясните разницу между `ChannelWriter<T>.WriteAsync(T item)` и `ChannelWriter<T>.TryWrite(T item)`. В каких ситуациях вы бы использовали каждый из них?
2. Зачем нужно вызывать `ChannelWriter<T>.Complete()`? Как это влияет на `ChannelReader<T>`?



Типы каналов и их конфигурация



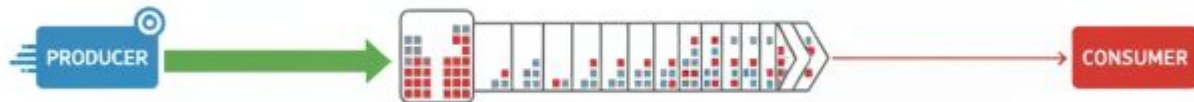
Неограниченные каналы (UnboundedChannel)

- **Создание:** `Channel.CreateUnbounded<T>()`.
- **Принцип работы:** Производитель может записывать элементы в канал всегда, без ожидания.
- **Преимущества:**
 - Простота: Самый легкий в использовании тип канала.
 - Отсутствие блокировок при записи: `WriteAsync()` всегда завершается мгновенно (в идеале, если нет других синхронизационных накладных расходов).
Производитель никогда не ждет.
 - Подходит для сценариев, где производитель генерирует данные нечасто или с равномерной скоростью, и потребление всегда опережает производство.



Потенциальные проблемы

- Неограниченное потребление памяти: Если производитель генерирует данные быстрее, чем потребитель успевает их обрабатывать, очередь будет бесконечно расти, потребляя все больше и больше памяти.
- Может привести к `OutOfMemoryException` в долгосрочной перспективе или при пиковых нагрузках.
- Отсутствие "обратного давления" (Backpressure): Производитель не знает, что потребитель не справляется, и продолжает генерировать данные.



Ограниченные каналы (BoundedChannel)

- **Создание:** `Channel.CreateBounded<T>(BoundedChannelOptions options)`.
- **Принцип работы:** Канал имеет фиксированный максимальный размер (Capacity).
- **Преимущества:**
 - Контроль над потреблением памяти: Гарантируется, что канал никогда не превысит заданное количество элементов.
 - Применение "обратного давления" (Backpressure): Когда канал достигает своей емкости, производитель будет вынужден ждать или использовать другую стратегию. Это сигнализирует производителю о замедлении.
 - Предсказуемость: Помогает стабилизировать систему при неравномерной нагрузке.



Ограниченные каналы (BoundedChannel)

- Недостатки:
 - Требуется более внимательного проектирования, так как необходимо выбирать стратегию поведения при заполнении.



BoundedChannelOptions

- **Capacity (обязательно):** Максимальное количество элементов, которое может хранить канал.
- **FullMode (обязательно):** Определяет, что происходит, когда производитель пытается записать элемент в полностью заполненный канал.
- **SingleReader / SingleWriter (необязательно):** Подсказывает каналу, что будет только один читатель/писатель. Позволяет внутренне оптимизировать производительность за счет пропуска некоторых блокировок. Используйте, только если это действительно так!
- **AllowSynchronousContinuations (необязательно):** По умолчанию false. Если true, продолжения Task могут быть запущены синхронно в потоке, который завершает операцию (например, читатель может запустить продолжение писателя). Может улучшить производительность, но требует осторожности во избежание дедлоков или неожиданного поведения.



BoundedChannelFullMode

- **Wait** (по умолчанию):
 - Производитель асинхронно ждет, пока место не освободится.
 - Классическое "обратное давление".
- **DropNewest:**
 - Когда канал полон, выбрасывается самый новый элемент, который пытается записать производитель.
 - Производитель не ждет, его элемент отбрасывается.
- **DropOldest:**
 - Когда канал полон, выбрасывается самый старый элемент из очереди, чтобы освободить место для нового.
 - Производитель не ждет, новый элемент записывается вместо старого.
- **DropWrite:**
 - Когда канал полон, попытка записи полностью игнорируется, новый элемент не добавляется.
 - WriteAsync завершается мгновенно, но элемент не попадает в канал.



Выбор правильного канала

Используйте `UnboundedChannel` когда:

- Объем данных небольшой или производительность потребителя гарантированно выше, чем у производителя.
- Вам нужна максимальная простота и не критично отслеживать объем очереди.
- Потеря данных абсолютно недопустима, и вы уверены, что не столкнетесь с `OutOfMemoryException`.

Используйте `BoundedChannel` когда:

- Необходимо контролировать потребление памяти очередью.
- Необходимо применить "обратное давление" на производителя (с `FullMode.Wait`).
- Допустима потеря данных (с `FullMode.DropNewest`, `DropOldest`, `DropWrite`) для поддержания актуальности или предотвращения задержек.
- Вы работаете с потоковыми данными, где старые данные могут устаревать (например, видеокадры, телеметрия).



ПРИМЕР

Вопросы для проверки

1. Какова основная разница между неограниченным (UnboundedChannel) и ограниченным (BoundedChannel) каналом? Какое потенциальное негативное следствие использования неограниченного канала в сценарии, где производитель работает значительно быстрее потребителя?
2. Перечислите как минимум две стратегии поведения при заполнении ограниченного канала (BoundedChannelFullMode) и кратко опишите, в каких ситуациях каждая из них может быть полезна.



Управление жизненным циклом канала и обработка завершения



Корректное закрытие канала с помощью Complete()

- Complete() сигнализирует, что больше данных не будет записано в канал.
- Это немедленно не закрывает канал, а лишь указывает на отсутствие новых элементов.
- Существующие элементы в буфере канала будут прочитаны потребителями.
- После вызова Complete(), попытки записи с помощью WriteAsync() или TryWrite() приведут к ошибке (ChannelClosedException или возврат false).
- Обязательно вызывать Complete() после того, как все производители закончили свою работу.



Как потребитель узнает о завершении

- `ChannelReader<T>.Completion Task`: Самый прямой способ. Task, который завершается, когда канал полностью прочитан и производитель вызвал `Complete()`. Если канал завершился с ошибкой, Task.Completion перейдёт в faulted состояние.
- `ReadAsync()` / `TryRead()` / `WaitToReadAsync()`:
 - После `Complete()`, если буфер пуст, `ReadAsync()` выдаст `ChannelClosedException`.
 - `TryRead()` вернёт `false`.
 - `WaitToReadAsync()` вернёт `false` (после того, как буфер опустеет).
- `await foreach (var item in reader.ReadAllAsync())`: Удобный синтаксис, который автоматически обрабатывает завершение. Цикл прекратится, когда канал опустеет и будет помечен как завершённый. Если канал завершился с ошибкой, `ReadAllAsync()` выбросит исключение.

ПРИМЕР

Обработка исключений в конвейере

- Исключения, возникающие в производителях, должны быть пойманы и, как правило, привести к вызову `writer.Complete(exception)`.
- Это помечает канал как завершённый с ошибкой, и `ChannelReader<T>.Completion` перейдёт в `faulted` состояние.
- `ReadAllAsync()` или `ReadAsync()` в потребителе выбросят это исключение (обёрнутое в `ChannelClosedException`).
- Исключения, возникающие в потребителях, обрабатываются как обычные исключения в асинхронных методах (`try-catch`). Они не влияют на состояние канала, если потребитель не вызовет `writer.Complete(exception)` (что не является стандартным сценарием).
- **Важно:** Исключение в одном производителе не должно "повесить" всю систему. Сигнализация через `Complete(exception)` позволяет другим компонентам адекватно отреагировать.

ПРИМЕР

Пример с множественными производителями и потребителями

Множественные производители:

- Все производители пишут в один и тот же `ChannelWriter<T>`.
- Только один производитель должен вызывать `Complete()` для канала.
- Предпочтительно использовать `Task.WhenAll()` для ожидания завершения всех производителей, а затем вызвать `channel.Writer.Complete()` после их завершения.

Множественные потребители:

- Все потребители читают из одного и того же `ChannelReader<T>`.
- Канал распределяет данные по мере доступности (fair-queueing).
- Когда канал завершается, все потребители получают сигнал о завершении (например, `ReadAllAsync()` завершит цикл).

ПРИМЕР

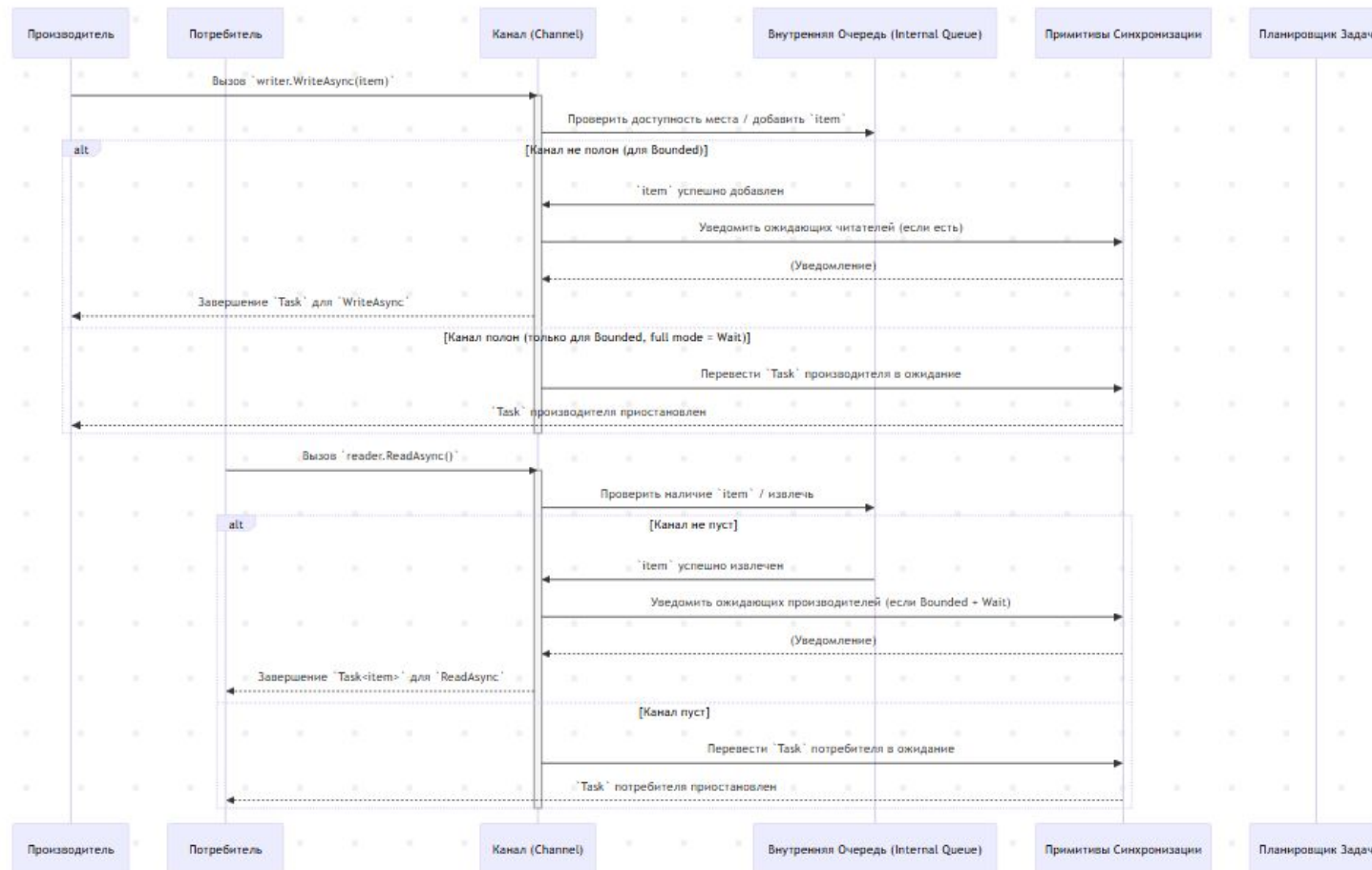
Вопросы для проверки

1. Помимо вызова `Complete()`, как можно узнать, что канал полностью завершил свою работу (т.е. был помечен как завершённый, и все элементы были прочитаны)? Какой API `ChannelReader<T>` для этого используется?
2. Что произойдет с `ChannelReader<T>.ReadAsync()`, если канал был завершён (`Complete()`) и все элементы из него уже прочитаны?

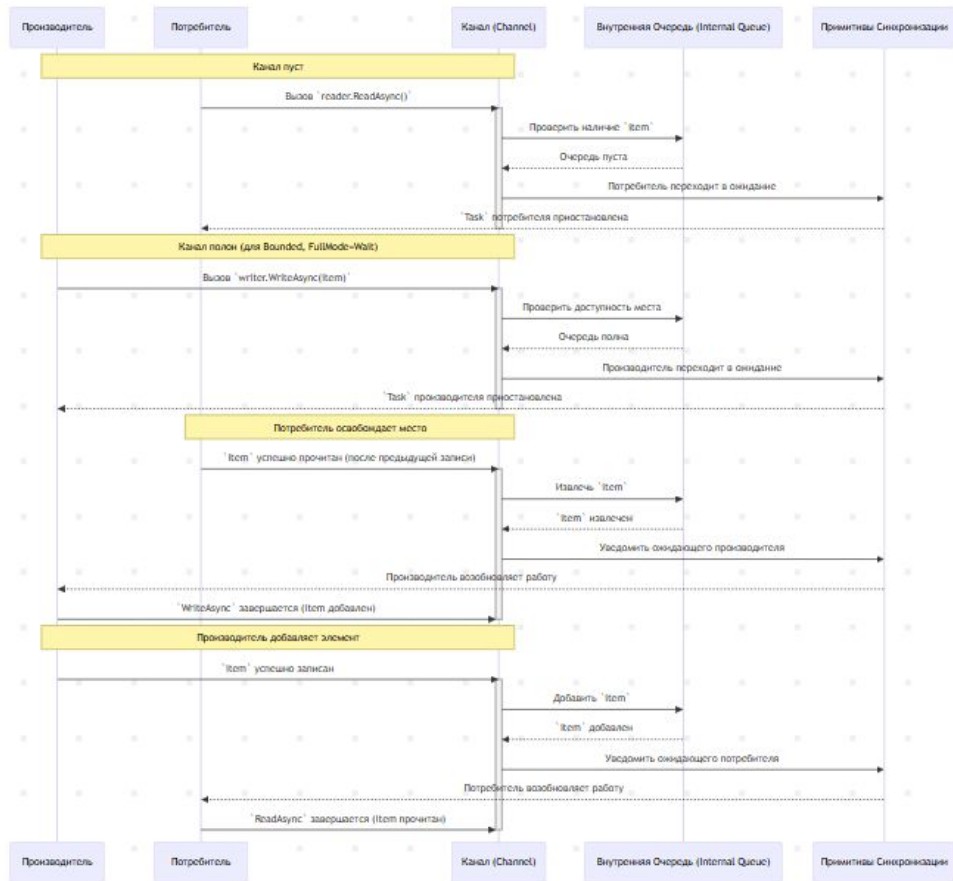


Подробности

Успешная запись и чтение



Блокировка/Ожидание Производителя и Потребителя



**Подробно о работе с памятью
внутри Channel**



UnboundedChannel<T>

- Если указать `SingleWriter = true` и `SingleReader = true`, реализация выберет внутреннюю очередь `SingleProducerSingleConsumerQueue<T>` (чрезвычайно лёгкую неблокируемую очередь SPSC).
- Если разрешено несколько писателей или несколько читателей, она возвращается к использованию `ConcurrentQueue<T>` для внутреннего хранения элементов.
- Вокруг этой очереди она поддерживает ожидающих (ожидающих чтения и записи) как лёгкие объекты `AsyncOperation` и использует `Interlocked`-операции для их возобновления.



UnboundedChannel<T>

- Внутренняя структура: Внутри UnboundedChannel обычно использует System.Collections.Concurrent.ConcurrentQueue<T>.
- Выделение памяти:
 - Когда вы создаете UnboundedChannel, выделяется небольшое количество памяти для самой ConcurrentQueue и сопутствующих объектов управления.
 - Для каждого элемента T, который записывается в канал:
 - Если T является ссылочным типом (класс), то выделяется память для самого объекта T (если он еще не был выделен до передачи в канал). ConcurrentQueue хранит ссылки на эти объекты.
 - Если T является значимым типом (структура), то копия этого типа будет сохранена в ConcurrentQueue. Это означает, что для каждого элемента структуры выделяется новая память внутри ConcurrentQueue.
 - ConcurrentQueue имеет внутреннюю структуру на основе связанных списков (или сегментированных массивов в более новых реализациях), где каждый узел/сегмент содержит несколько элементов. По мере добавления элементов, ConcurrentQueue динамически расширяет свою внутреннюю структуру, выделяя новые узлы/сегменты.

UnboundedChannel<T>

Освобождение памяти:

- Когда элемент T извлекается потребителем, он удаляется из ConcurrentQueue.
- Если T был ссылочным типом, то ссылка удаляется из очереди. Сам объект T может быть собран сборщиком мусора (GC), если на него больше нет ссылок нигде.
- Если T был значимым типом, то память, занимаемая его копией внутри ConcurrentQueue, становится доступной для GC.
- Внутренние структуры ConcurrentQueue могут быть переиспользованы или освобождены, но это обычно происходит менее агрессивно, чем с элементами.

Проблемы с памятью:

- Основная проблема - неконтролируемый рост памяти. Если производитель постоянно добавляет элементы быстрее, чем потребитель их обрабатывает, ConcurrentQueue будет продолжать расти, пока не исчерпает доступную память, что приведет к OutOfMemoryException.
- Это особенно критично для значимых типов, так как каждая запись создает новую копию.



BoundedChannel<T>

- Всегда использует настраиваемый кольцевой буфер (массив T) с индексами начала/конца (mod capacity). Это НЕ Queue<T>; это более компактная структура, доступная благодаря знанию фиксированной ёмкости.
- Синхронизация в основном выполняется с помощью операций Interlocked/Volatile, а в некоторых многопользовательских случаях — лёгкой блокировки (монитор) для координации ожидающих.
- Полные режимы (Wait, DropWrite, DropNewest, DropOldest) обрабатываются явно в этой логике; ни один из существующих типов очередей BCL напрямую не поддерживает эти политики.

BoundedChannel<T>

Контроль памяти:

- Главное преимущество BoundedChannel - ограниченный и контролируемый расход памяти. Максимальный объем памяти, занимаемый элементами в канале, жестко ограничен capacity.
- Если канал полон и производитель пытается записать новый элемент в режиме Wait, производитель будет заблокирован, предотвращая дальнейшее выделение памяти для элементов канала.
- Режимы DropNewest/DropOldest также помогают управлять памятью, отбрасывая элементы вместо блокировки или переполнения.
- Не создаются объекты Task для каждого элемента; асинхронные ожидания представляют собой облегченные структуры с ожиданием, объединенные в пул.



Сравнение с другими конструкциями

- **Queue<T>**: простой FIFO, не потокобезопасный, без асинхронности, без семантики завершения.
- **ConcurrentQueue<T>**: потокобезопасный FIFO, но без асинхронного ожидания или завершения/распространения ошибок.
- **BlockingCollection<T>**: может обертывать ConcurrentQueue или другие; обеспечивает блокировочную (не асинхронную) семантику и отмену, но меньше политик и больше накладных расходов на асинхронное соединение.
- **TPL Dataflow**: расширенные возможности графов/сообщений, планирование, блоки выполнения; более выраженная абстракция. Каналы стремятся быть минимальными примитивами.

Ключевые тезисы

- Ссылочные типы (class): Канал хранит только ссылки. Объем памяти, занимаемый самими объектами, зависит от того, сколько на них ссылок в целом. Канал лишь добавляет/удаляет одну из таких ссылок.
- Значимые типы (struct): Канал хранит копии структур. Это означает, что при записи каждой структуры в канал создается новая копия, и именно эта копия занимает место в памяти канала. Если структуры большие, это может существенно увеличить потребление памяти.
- Внутренние механизмы синхронизации: Каналы также используют небольшое количество памяти для внутренних примитивов синхронизации (мьютексы, семафоры, объекты TaskCompletionSource и т.д.), которые управляют ожиданием и уведомлением производителей и потребителей.



Важный вывод о памяти:

Если вам нужно жестко контролировать потребление памяти, всегда используйте BoundedChannel. Если вы уверены, что потребитель всегда будет успевать обрабатывать данные, или если кратковременные пики потребления памяти допустимы, UnboundedChannel может быть проще в использовании, но всегда помните о риске OutOfMemoryException.

Сценарии и интеграция с System.IO.Pipelines

Сценарии использования

- **Обработка запросов:** Отправка входящих запросов (веб, RPC) в фоновую очередь для асинхронной обработки, разгружая основной поток запросов. Пример: ASP.NET Core контроллер принимает запрос, помещает его в Channel, возвращает Accepted, а фактическая работа происходит в фоне.
- **Фоновые задачи:** Управление асинхронными процессами, обработка событий, отправка уведомлений, длительные вычисления. Пример: Сервис мониторинга принимает события, помещает их в Channel, отдельный набор воркеров обрабатывает их (например, отправляет Email/SMS).
- **Системы логирования:** Асинхронная запись логов без блокировки основного приложения. Пример: Приложение пишет сообщения в Channel<LogEntry>, отдельный фоновый поток читает из канала и записывает в файл/БД/удаленный сервис.
- **Парсинг потоковых данных:** Эффективная обработка данных, поступающих непрерывным потоком (файлы, сеть), с разделением этапов чтения и парсинга. Пример: Чтение большого CSV файла по частям, парсинг каждой части и передача готовых объектов для дальнейшей обработки.



System.IO.Pipelines

- Назначение: Оптимизирован для обработки потоков байтов (ReadOnlySequence<byte>).
- Сфокусирован на:
 - а. Эффективном управлении памятью (пулы буферов MemoryPool<byte>).
 - б. Минимизации копирования данных (использование ReadOnlySequence<byte>).
 - в. Поддержке асинхронного чтения/записи низкоуровневых бинарных данных, таких как сетевые потоки, файловые потоки и т.д.
- Что делает: Предоставляет API для эффективного чтения и записи сырых байтов, позволяя парсить их без создания лишних копий и аллокаций. Отлично подходит для реализации сетевых протоколов, парсеров файлов и других задач, где важна работа с байтовыми буферами.
- Типичный сценарий: Сервер, который принимает байты по TCP/IP, парсит их в HTTP-запросы (или другой протокол). Здесь Pipe поможет эффективно управлять буферами, чтобы не копировать данные по мере их чтения и парсинга.
- Ключевая идея: Эффективная работа с сырыми байтами и управлением буферами памяти.

System.Threading.Channels

- Назначение: Оптимизирован для передачи типизированных объектов (T) между асинхронными компонентами.
- Сфокусирован на:
 - a. Реализации паттерна "Производитель-Потребитель" для объектов.
 - b. Обеспечении асинхронной очереди для любых .NET объектов.
 - c. Предоставлении гибких стратегий для управления потоком (ограниченные/неограниченные каналы, режимы переполнения).
- Что делает: Служит как асинхронная, потокобезопасная очередь, где производители помещают объекты типа T, а потребители их извлекают. Он не занимается напрямую управлением буферами памяти для сырых байтов, а скорее управляет очередью ссылок на объекты.
- Типичный сценарий: Веб-сервер, который после парсинга HTTP-запроса (возможно, с помощью Pipelines) создает объект HttpRequest и помещает его в Channel<HttpRequest>, откуда фоновые воркеры асинхронно забирают и обрабатывают эти запросы.
- Ключевая идея: Эффективная асинхронная передача типизированных объектов между задачами/потоками.

Когда что использовать?

Используйте `System.IO.Pipelines` когда:

- Вы работаете с сырыми байтами (например, с сетевыми или файловыми потоками).
- Вам нужна максимальная производительность при чтении/записи байтов, минимизация аллокаций и копирований.
- Вы реализуете протокол или парсер, который преобразует байты в структурированные данные.
- Пример: разработка собственного веб-сервера, высокопроизводительного файлового конвейера, обработчика бинарных данных.

Используйте `System.Threading.Channels` когда:

- Вы хотите передавать логические, уже обработанные объекты (не сырые байты) между асинхронными компонентами.
- Вам нужно реализовать паттерн "Производитель-Потребитель" для задач, сообщений, событий.
- Вы хотите развязать компоненты системы, чтобы они могли работать с разной скоростью.
- Вам нужна асинхронная очередь с возможностью ограничения размера и управления поведением при переполнении.
- Пример: очередь HTTP-запросов для фоновой обработки, очередь задач для пула воркеров, конвейер обработки данных.

Пример совместного использования (Pipelines + Channels)

- Сетевой ввод: Сервер принимает байты из TCP-соединения.
 - Здесь `System.IO.Pipelines` незаменим. `PipeReader` эффективно читает байты из сокета, не копируя их.
- Парсинг протокола: `PipeReader` используется для парсинга этих сырых байтов в логические "сообщения" или "запросы" (например, HTTP-заголовки + тело).
 - Все еще `System.IO.Pipelines` для эффективного доступа к байтовым буферам.
- Обработка бизнес-логики: Как только байты успешно спарсены в объект (например, `HttpRequest` или `MyCustomMessage`), этот объект нужно передать на дальнейшую обработку.
 - Здесь `System.Threading.Channels`. Созданный объект (`HttpRequest`) помещается в `Channel<HttpRequest>`, откуда пул асинхронных воркеров забирает его для выполнения бизнес-логики.
 - Это позволяет развязать I/O-поток от потока обработки логики, обеспечивая максимальную отзывчивость и масштабируемость.

ПРИМЕР

Вопросы



если есть вопросы



если вопросов нет

**Подведём
итоги занятия**

Ключевые тезисы занятия

- Channels (System.Threading.Channels): это высокопроизводительный, асинхронный и потокобезопасный механизм для обмена данными между производителями (producers) и потребителями (consumers) в .NET.
- Producer/Consumer паттерн: Каналы являются идеальным решением для реализации этого паттерна, позволяя одному или нескольким производителям отправлять данные, а одному или нескольким потребителям их получать, работая асинхронно и независимо друг от друга.
- Типы каналов:
- UnboundedChannel: канал без ограничений по размеру. Потребляет больше памяти, но никогда не блокирует производителя.
- BoundedChannel: канал с фиксированным размером. Если канал полон, производитель может быть заблокирован (в зависимости от full mode), что помогает контролировать потребление ресурсов.



Ключевые тезисы занятия

- `ChannelWriter<T>`: используется для записи элементов в канал (метод `WriteAsync`, `TryWrite`).
- `ChannelReader<T>`: используется для чтения элементов из канала (методы `ReadAsync`, `TryRead`, `ReadAllAsync`).
- Асинхронность: Каналы полностью асинхронны и прекрасно интегрируются с `async/await`, что упрощает работу с операциями ввода/вывода и параллелизм.
- Завершение канала (`Complete`): Важно вызывать `writer.Complete()` (или `writer.TryComplete()`), чтобы сигнализировать читателям, что больше данных не поступит. Это позволяет читателям корректно завершить свою работу.
- Backpressure: В `BoundedChannel` с соответствующим `BoundedChannelFullMode` (например, `Wait`), каналы обеспечивают механизм "обратного давления" (`backpressure`), замедляя производителей, если потребители не успевают обрабатывать данные.



Ключевые тезисы занятия

Каналы отлично подходят для:

- Обработки потоков данных (например, логов, событий).
- Реализации конвейеров данных.
- Очередей сообщений внутри одного приложения.
- Задач, где есть медленный потребитель и быстрый производитель.

Преимущества перед другими механизмами:

- Более современный и гибкий, чем `BlockingCollection<T>`.
- Проще в использовании для асинхронных сценариев, чем низкоуровневые примитивы синхронизации.
- Встроенная поддержка асинхронности и `async/await`.



Цели вебинара

К концу занятия вы сможете

1. Реализовывать паттерн "Производитель-потребитель" с помощью Channels;
2. Строить эффективные асинхронные конвейеры для разделения I/O-операций и логики обработки;
3. Управлять жизненным циклом канала и обрабатывать его закрытие.





**Отправьте в чат эмодзи, который
отражает ваше настроение
после занятия**



**Продолжите высказывание: теперь
я умею/знаю...**



**Что планируете использовать
в работе?**

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

